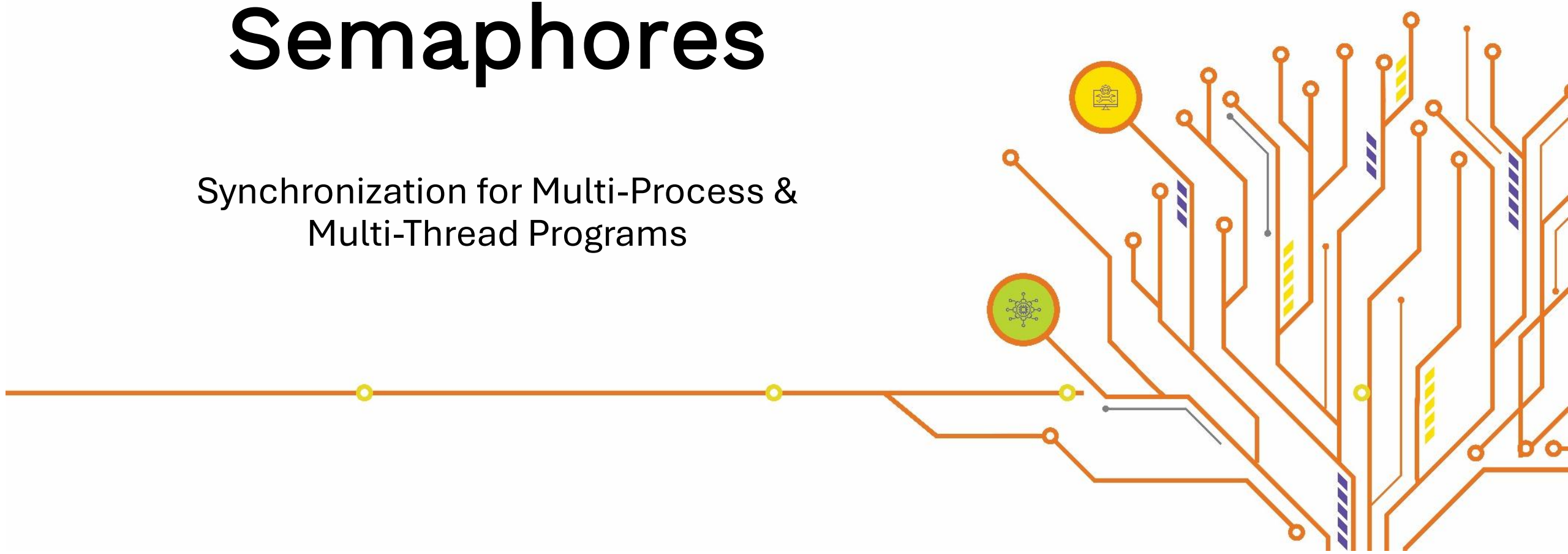
KPIT | APEX LAB

POSIX Semaphores

Synchronization for Multi-Process & Multi-Thread Programs



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

What is a Semaphore?

A synchronization primitive

- Manages access to shared resources
- Based on a counter
- Processes/threads wait when counter = 0
- Used for:
  - mutual exclusion
  - producer–consumer
  - signaling events
  - limiting access to N resources

Counter ↓ as tasks enter
↑ as they leave

Types of POSIX Semaphores

Unnamed Semaphores	Named Semaphores
• Stored in memory • Used between threads or processes (with shared memory) • Created using sem_init()	• Identified by a name (e.g., /semApex) • Work across unrelated processes • Created using sem_open()

Semaphore Operations

- sem_wait()
  - Decrements counter
  - Blocks if counter = 0
- sem_post()
  - Increments counter
  - Wakes waiting process/thread
- sem_getvalue()
  - Read current count (debug/testing)

Unnamed Semaphore (sem_init)

- Prototype:
  - int sem_init(sem_t *sem, int pshared, unsigned int value);
  - Fields:
    - pshared = 0 → between threads
    - pshared = 1 → between processes (in shared memory)
    - value → initial count
- Example:
  - sem_t lock;sem_init(&lock, 0, 1); // binary semaphore

Named Semaphores (sem_open)

- Prototype:
  - sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
- Usage:
  - sem_t *s = sem_open("/apexsem", O_CREAT, 0666, 1);
- Notes:
  - Name must begin with /
  - Exists until removed using sem_unlink()
  - Works between unrelated processes

Binary vs Counting Semaphores

Binary Semaphores	Counting Semaphores
• Value = 1 • Used like a mutex • Ensures exclusive access	• Value = N • Allows up to N simultaneous accesses • Used for resource pools, producer–consumer

Example (Binary Semaphore — Mutual Exclusion)

```
sem_t lock;

void *worker(void *arg) {
   sem_wait(&lock);
   // critical section
   sem_post(&lock);
}
```

- Use case:
  - Protect shared variable
  - Protect file writes / hardware access
  - Avoid race conditions

Semaphore Lifecycle

Create: sem_init() → Use: sem_wait() / sem_post() → Destroy: sem_destroy()

Create/Open: sem_open() → Use: sem_wait() / sem_post() → Close: sem_close() → Remove: sem_unlink("/semApex")

Blocking vs Non-Blocking

- sem_wait():
  - blocks until counter > 0
- sem_trywait():
  - Returns immediately
  - Fails with EAGAIN if count = 0
- Useful for:
  - Real-time / non-blocking tasks
  - Avoiding potential deadlocks

Initialization in Shared Memory (Inter-Process)

Create shared memory → Place sem_t inside shared region → Call sem_init(&sem, 1, initial_value); → All processes map same region and use semaphore

Ordering Constraints

Semaphores guarantee:	Do NOT guarantee:
• atomic decrement/increment • correct wake-up order (FIFO-ish, implementation-dependent) • protection of critical sections	• fairness • priority scheduling • starvation prevention

When to Use a Semaphore

- Sync between threads or processes
- Producer–consumer buffering
- Resource pool management
- Wake-up signaling
- Guarding critical regions
- Rate limiting (e.g., max N tasks allowed)

When NOT to Use Semaphores

- To protect complex data requiring ownership semantics → use mutex
- For waking specific processes → use condition variables
- For message-based communication → use MQ or pipes
- For large data transfer → use shared memory + semaphores

Deadlocks & Pitfalls

Avoid:

- Taking semaphores in different orders
- Forgetting sem_post()
- Waiting twice without posting
- Misplacing semaphores inside unmapped memory
- Using named semaphores without unlinking → leftover objects

Debugging & Observing Semaphores

- Check /dev/shm for named semaphores (Linux)
- Use strace to monitor sem_wait() / sem_post()
- Check permissions with ls -l /dev/shm/sem.
- Ensure proper unlink on exit

Summary

- Semaphores synchronize processes/threads using a counter
- Binary → mutual exclusion
- Counting → resource management
- POSIX APIs: sem_init, sem_open, sem_wait, sem_post
- Must use carefully to avoid deadlocks
- Essential in embedded systems for safe concurrency

Scenario: Parking Lot Capacity Controller (1/3)

- You're simulating a mall parking lot with limited slots (N).
- Each arriving car is a thread that tries to enter, parks for a while, and leaves.
- A counting semaphore enforces capacity so that at most N cars are parked simultaneously.
- A binary semaphore (or mutex) protects shared logs to prevent interleaved prints.

Scenario: Parking Lot Capacity Controller (2/3)

PART 1 — Core Capacity Control

- Inputs (CLI): N_slots (e.g., 5), N_cars (e.g., 20).
- Initialize counting semaphore to N_slots.
- Spawn N_cars threads; each thread:
  - Prints Car <id> arriving
  - sem_wait(&slots) → blocks if full
  - Prints Car <id> parked
  - Sleeps 1–3 sec (random)
  - Prints Car <id> leaving
  - sem_post(&slots)
  - Use a mutex/binary semaphore around printf to avoid mixing lines.

Scenario: Parking Lot Capacity Controller (3/3)

PART 2 — Arrival Patterns & Metrics

- Randomize inter-arrival times (0–500 ms).
- Collect metrics: peak waiters, avg wait time before getting a slot.
- Print a final summary table.